Git/GitHub Documentation: Remote

Gagan Pradhan

January 4, 2026

1 Introduction: Online Backup & GitHub

1.1 Online Backup Systems

An effective online backup system typically involves:

1. **Online Backup:** Storing data in the cloud.
2. **Sync Updates (Computer → Drive):** Uploading local changes.
3. **Sync Updates (Drive → Computer):** Downloading remote changes.

Points 2 and 3 together constitute **2-Way Sync**.

1.2 Why GitHub for Code?

While services like Google Drive are excellent for documents, they are not designed to track complex changes in a code workspace (lines changed, deletions, versions).

GitHub is specifically designed to store, manage, and organize **Git repositories**, providing version control features alongside storage.

2 Feature 1: Creating an Online Backup

To backup code, we link our **Local Repository** (on your computer) to a **Remote Repository** (on GitHub).

2.1 Step-by-Step Setup

1. **Create a Remote Repo:** Create a new empty repository on GitHub.
2. **Link Local to Remote:** Use the terminal in your project path.

```
git remote add origin <URL>
```

   Note: 'origin' is the standard alias name for the remote repository URL.

3. **Verify Connection:**

```
# Simple check
git remote

# Verbose check (shows URLs)
git remote -v
```

4. **Configure Credentials:** Tell Git who you are for authentication.

```
git config --global credential.username "skysys007"
```

5. **Upload (Push) Code:**

```
git push origin main
```

   This pushes the code from your local 'main' branch to the 'origin' remote.

2.2 Removal

If you need to unlink a remote:

```
git remote remove origin
```

3 Feature 2: Sync Changes (Computer → GitHub)

When working locally, your history may diverge from the remote server. Git tracks this using two branch pointers:

- **main:** Your local current state.
- **origin/main:** The state of the code on GitHub (last time you checked).

3.1 Visualizing Sync Status

Use the graph command to see the relationship between local and remote.

```
git log --all --graph
```

Scenario 1: Synced

```
* commit a596... (HEAD -> main, origin/main)
```

Both branches are at the same commit.

Scenario 2: Local Ahead (Changes made locally) If you commit changes locally, ‘main‘ moves forward, but ‘origin/main‘ stays behind.

```
git commit -m "Made Changes in the readme file"
```

Log Output:

```
* commit 4448... (HEAD -> main)  <-- Local is ahead
|    Made Changes in the readme file
|
* commit a596... (origin/main)   <-- Remote is behind
```

To sync them, you must **Push**:

```
git push origin main
```

3.2 Upstream Shortcut

To avoid typing ‘origin main‘ every time, set the upstream default:

```
git push -u origin main
# OR
git push --set-upstream origin main
```

Now you can simply use `git push`.

3.3 Amending & Force Push

Warning: If you modify history using ‘–amend‘ *after* you have already pushed, your local history will diverge from the remote history.

The Problem:

```
* commit 593f... (HEAD -> master)  <-- Your new amended commit
|
| * commit 9674... (origin/master) <-- The old commit on GitHub
|/
```

Git will block a standard push because the histories do not match.

The Solution: You must force the remote to accept your new history, deleting the old commit on GitHub.

```
git push -f origin master
```

4 Feature 3: Sync Changes (GitHub → Computer)

This workflow handles updates made by others or from different machines.

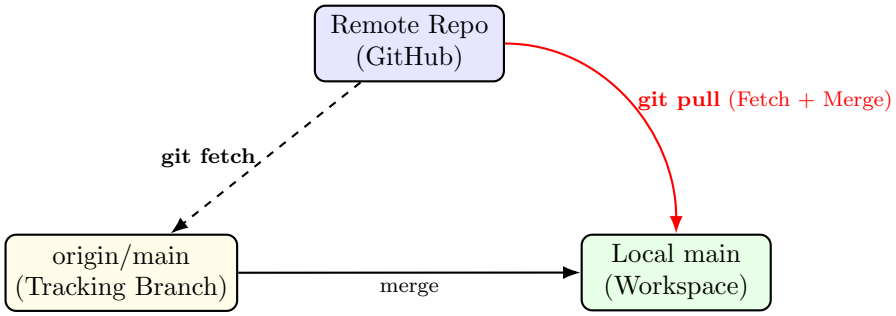
4.1 Cloning (First Time Download)

Copies a remote repository to your local system.

```
git clone <URL> <folder_name>
```

Note: This creates a separate instance of the repo. Changes here do not automatically affect the original folder unless pushed.

4.2 Fetching vs. Pulling (Updating Existing Repo)



4.2.1 1. git fetch

Updates your remote-tracking branches (like ‘origin/main‘) to reflect the current state on GitHub, but **does not touch your working files**.

```
git fetch
```

After fetching, ‘git log –all –graph‘ will show that ‘origin/main‘ is ahead of your local ‘main‘.

4.2.2 2. git pull

Fetches the updates **AND** merges them into your local branch immediately.

```
git pull origin main
```

This syncs your local repo with the remote.

4.3 Pull Shortcut

Like push, you can configure the default upstream branch for pulling:

```
git branch --set-upstream-to=origin/main main
```

Now you can simply use `git pull`.